# Configuring projects

## Python version requirement

Projects may declare the Python versions supported by the project in the `project.requires-python` field of the `pyproject.toml`.

It is recommended to set a `requires-python` value:

```
toml title="pyproject.toml" hl_lines="4" [project] name = "example" version = "0.1.0"
requires-python = ">=3.12"
```

The Python version requirement determines the Python syntax that is allowed in the project and affects selection of dependency versions (they must support the same Python version range).

## Entry points

Entry points are the official term for an installed package to advertise interfaces. These include:

- Command line interfaces
- Graphical user interfaces
- Plugin entry points

!!! important

```
Using the entry point tables requires a [build system](#build-systems) to be defined.
```

### Command-line interfaces

Projects may define command line interfaces (CLIs) for the project in the `[project.scripts]` table of the `pyproject.toml`.

For example, to declare a command called `hello` that invokes the `hello` function in the `example` module:

```
toml title="pyproject.toml" [project.scripts] hello = "example:hello"
```

Then, the command can be run from a console:

```
$ uv run hello
```

### Graphical user interfaces

Projects may define graphical user interfaces (GUIs) for the project in the `[project.gui-scripts]` table of the `pyproject.toml`.

!!! important

```
These are only different from [command-line interfaces](#command-line-interfaces) on
Windows, where
they are wrapped by a GUI executable so they can be started without a console. On
other platforms,
they behave the same.
```

For example, to declare a command called `hello` that invokes the `app` function in the `example` module:

```
toml title="pyproject.toml" [project.gui-scripts] hello = "example:app"
```

### Plugin entry points

Projects may define entry points for plugin discovery in the `[project.entry-points]` table of the `pyproject.toml`.

For example, to register the `example-plugin-a` package as a plugin for `example`:

```
toml title="pyproject.toml" [project.entry-points.'example.plugins'] a =
"example_plugin_a"
```

Then, in `example`, plugins would be loaded with:

```python title="example/__init__.py"
from importlib.metadata import entry_points

for plugin in entry_points(group="example.plugins"):
    plugin.load()
```

```
!!! note

    The `group` key can be an arbitrary value, it does not need to include the
package name or
    "plugins". However, it is recommended to namespace the key by the package name to
avoid
    collisions with other packages.

## Build systems

A build system determines how the project should be packaged and installed. Projects
may declare and
configure a build system in the `[build-system]` table of the `pyproject.toml`.

uv uses the presence of a build system to determine if a project contains a package
that should be
installed in the project virtual environment. If a build system is not defined, uv
will not attempt
to build or install the project itself, just its dependencies. If a build system is
defined, uv will
build and install the project into the project environment.

By default, `uv init` creates a packaged project using the [uv build backend](../
build-backend.md).
The `--build-backend` option can be provided to select an alternative build backend,
and
`--no-package` can be provided to create a flat, unpackaged project instead.

!!! note

    While uv will not build and install the current project without a build system
definition,
    the presence of a `[build-system]` table is not required in other packages. For
legacy reasons,
    if a build system is not defined, then `setuptools.build_meta:__legacy__` is used
to build the
    package. Packages you depend on may not explicitly declare their build system but
are still
    installable. Similarly, if you [add a dependency on a local project](./
dependencies.md#path)
    or install it with `uv pip`, uv will attempt to build and install it regardless
of the presence
    of a `[build-system]` table.

Build systems are used to power the following features:

- Including or excluding files from distributions
- Editable installation behavior
```

- Dynamic project metadata
- Compilation of native code
- Vendoring shared libraries

To configure these features, refer to the documentation of your chosen build system.

## Project packaging

As discussed in [build systems](#build-systems), a Python project must be built to be installed.
This process is generally referred to as "packaging".

You probably need a package if you want to:

- Add commands to the project
- Distribute the project to others
- Use a `src` and `test` layout
- Write a library

You probably _do not_ need a package if you are:

- Writing scripts
- Building a simple application
- Using a flat layout

While uv usually uses the declaration of a [build system](#build-systems) to determine if a project
should be packaged, uv also allows overriding this behavior with the
[`tool.uv.package`](../../reference/settings.md#package) setting.

Setting `tool.uv.package = true` will force a project to be built and installed into the project
environment. If no build system is defined, uv will use the setuptools legacy backend.

Setting `tool.uv.package = false` will force a project package _not_ to be built and installed into
the project environment. uv will ignore a declared build system when interacting with the project;
however, uv will still respect explicit attempts to build the project such as invoking `uv build`.

## Project environment path

The `UV_PROJECT_ENVIRONMENT` environment variable can be used to configure the project virtual
environment path (`.venv` by default).

If a relative path is provided, it will be resolved relative to the workspace root. If an absolute
path is provided, it will be used as-is, i.e., a child directory will not be created for the
environment. If an environment is not present at the provided path, uv will create it.

This option can be used to write to the system Python environment, though it is not

```
recommended.
`uv sync` will remove extraneous packages from the environment by default and, as
such, may leave
the system in a broken state.

To target the system environment, set `UV_PROJECT_ENVIRONMENT` to the prefix of the
Python
installation. For example, on Debian-based systems, this is usually `/usr/local`:

```console
$ python -c "import sysconfig; print(sysconfig.get_config_var('prefix'))"
/usr/local
```

To target this environment, you'd export `UV_PROJECT_ENVIRONMENT=/usr/local`.

!!! important

```
If an absolute path is provided and the setting is used across multiple projects, the
environment will be overwritten by invocations in each project. This setting is only
recommended
for use for a single project in CI or Docker images.
```

!!! note

```
By default, uv does not read the `VIRTUAL_ENV` environment variable during project
operations.
A warning will be displayed if `VIRTUAL_ENV` is set to a different path than the
project's
environment. The `--active` flag can be used to opt-in to respecting `VIRTUAL_ENV`.
The
`--no-active` flag can be used to silence the warning.
```

## Build isolation

By default, uv builds all packages in isolated virtual environments alongside their declared build dependencies, as per PEP 517.

Some packages are incompatible with this approach to build isolation, be it intentionally or unintentionally.

For example, packages like `flash-attn` and `deepspeed` need to build against the same version of PyTorch that is installed in the project environment; by building them in an isolated environment, they may inadvertently build against a different version of PyTorch, leading to runtime errors.

In other cases, packages may accidentally omit necessary dependencies in their declared build dependency list. For example, `cchardet` requires `cython` to be installed in the project environment prior to installing `cchardet`, but does not declare it as a build dependency.

To address these issues, uv supports two separate approaches to modifying the build isolation behavior:

1. **Augmenting the list of build dependencies**: This allows you to install a package in an isolated environment, but with additional build dependencies that are not declared by the package itself via the `extra-build-dependencies` setting. For packages like `flash-attn`, you can even enforce that those build dependencies (like `torch`) match the version of the package that is or will be installed in the project environment.
2. **Disabling build isolation for specific packages**: This allows you to install a package without building it in an isolated environment.
```

When possible, we recommend augmenting the build dependencies rather than disabling build isolation entirely, as the latter approach requires that the build dependencies are installed in the project environment *prior* to installing the package itself, which can lead to more complex installation steps, the inclusion of extraneous packages in the project environment, and difficulty in reproducing the project environment in other contexts.

**Augmenting build dependencies**
To augment the list of build dependencies for a specific package, add it to the `extra-build-dependencies` list in your `pyproject.toml`.

For example, to build `cchardet` with `cython` as an additional build dependency, include the following in your `pyproject.toml`:

```toml title="pyproject.toml" [project] name = "project" version = "0.1.0" description = "..." readme = "README.md" requires-python = ">=3.12" dependencies = ["cchardet"]

[tool.uv.extra-build-dependencies] cchardet = ["cython"]


To ensure that a build dependency matches the version of the package that is or will be installed in
the project environment, set `match-runtime = true` in the `extra-build-dependencies` table. For
example, to build `deepspeed` with `torch` as an additional build dependency, include the following
in your `pyproject.toml`:

```toml title="pyproject.toml"
[project]
name = "project"
version = "0.1.0"
description = "..."
readme = "README.md"
requires-python = ">=3.12"
dependencies = ["deepspeed", "torch"]

[tool.uv.extra-build-dependencies]
deepspeed = [{ requirement = "torch", match-runtime = true }]
```

This will ensure that `deepspeed` is built with the same version of `torch` that is installed in the project environment.

!!! tip

```
Pre-built `deepspeed` wheels are also available from the
[Astral GPU indexes](../../guides/integration/pytorch.md#installing-gpu-enabled-pytorch-extensions).
```

Similarly, to build `flash-attn` with `torch` as an additional build dependency, include the following in your `pyproject.toml`:

```toml title="pyproject.toml" [project] name = "project" version = "0.1.0" description = "..." readme = "README.md" requires-python = ">=3.12" dependencies = ["flash-attn", "torch"]

[tool.uv.extra-build-dependencies] flash-attn = [{ requirement = "torch", match-runtime = true }]

[tool.uv.extra-build-variables] flash-attn = { FLASH_ATTENTION_SKIP_CUDA_BUILD = "TRUE" }
```
```

````
!!! note

    The `FLASH_ATTENTION_SKIP_CUDA_BUILD` environment variable enables `flash-attn`
to be
    resolved from a pre-built wheel, rather than attempting to build it from source,
which
    requires access to the CUDA development toolkit.

    If the CUDA toolkit is available during resolution, we recommend omitting the
    `FLASH_ATTENTION_SKIP_CUDA_BUILD` variable, as setting
`FLASH_ATTENTION_SKIP_CUDA_BUILD`
    to `TRUE` can lead to an incompatible install if no compatible pre-built wheel is
available
    for the target PyTorch version, GPU version, and platform.

!!! tip

    Pre-built `flash-attn` wheels are also available from the
    [Astral GPU indexes](../../guides/integration/pytorch.md#installing-gpu-enabled-
pytorch-extensions).

Similarly, [`deep_gemm`](https://github.com/deepseek-ai/DeepGEMM) follows the same
pattern:

```toml title="pyproject.toml"
[project]
name = "project"
version = "0.1.0"
description = "..."
readme = "README.md"
requires-python = ">=3.12"
dependencies = ["deep_gemm", "torch"]

[tool.uv.sources]
deep_gemm = { git = "https://github.com/deepseek-ai/DeepGEMM" }

[tool.uv.extra-build-dependencies]
deep_gemm = [{ requirement = "torch", match-runtime = true }]
````

!!! tip

````
Pre-built `deep_gemm` wheels are also available from the
[Astral GPU indexes](../../guides/integration/pytorch.md#installing-gpu-enabled-
pytorch-extensions).
````

The use of `extra-build-dependencies` and `extra-build-variables` are tracked in the uv cache, such that changes to these settings will trigger a reinstall and rebuild of the affected packages. For example, in the case of `flash-attn`, upgrading the version of `torch` used in your project would subsequently trigger a rebuild of `flash-attn` with the new version of `torch`.

**Dynamic metadata**

The use of `match-runtime = true` is only available for packages like `flash-attn` that declare static metadata. If static metadata is unavailable, uv is required to build the package during the dependency resolution phase; as such, uv cannot determine the version of the build dependency that would ultimately be installed in the project environment.
````

In other words, if `flash-attn` did not declare static metadata, uv would not be able to determine the version of `torch` that would be installed in the project environment, since it would need to build `flash-attn` prior to resolving the `torch` version.

As a concrete example, `axolotl` is a popular package that requires augmented build dependencies, but does not declare static metadata, as the package's dependencies vary based on the version of `torch` that is installed in the project environment. In this case, users should instead specify the exact version of `torch` that they intend to use in their project, and then augment the build dependencies with that version.

For example, to build `axolotl` against `torch==2.6.0`, include the following in your `pyproject.toml`:

```toml title="pyproject.toml" [project] name = "project" version = "0.1.0" description = "..." readme = "README.md" requires-python = ">=3.12" dependencies = ["axolotl[deepspeed, flash-attn]", "torch==2.6.0"]

[tool.uv.extra-build-dependencies] axolotl = ["torch==2.6.0"] deepspeed = ["torch==2.6.0"] flash-attn = ["torch==2.6.0"]


Similarly, older versions of `flash-attn` did not declare static metadata, and thus would not have
supported `match-runtime = true` out of the box. Unlike `axolotl`, though, `flash-attn` did not vary
its dependencies based on dynamic properties of the build environment. As such, users could instead
provide the `flash-attn` metadata upfront via the
[`dependency-metadata`](../../reference/settings.md#dependency-metadata) setting, thereby forgoing
the need to build the package during the dependency resolution phase. For example, to provide the
`flash-attn` metadata upfront:

```toml title="pyproject.toml"
[[tool.uv.dependency-metadata]]
name = "flash-attn"
version = "2.6.3"
requires-dist = ["torch", "einops"]
```

!!! tip

To determine the package metadata for a package like `flash-attn`, navigate to the appropriate Git repository,
or look it up on [PyPI](https://pypi.org/project/flash-attn) and download the package's source distribution.
The package requirements can typically be found in the `setup.py` or `setup.cfg` file.

(If the package includes a built distribution, you can unzip it to find the `METADATA` file; however, the presence
of a built distribution would negate the need to provide the metadata upfront, since it would already be available
to uv.)

The `version` field in `tool.uv.dependency-metadata` is optional for registry-based dependencies (when omitted, uv will assume the metadata applies to all versions of
```

```
the package),
but _required_ for direct URL dependencies (like Git dependencies).
```

**Disabling build isolation**

Installing packages without build isolation requires that the package's build dependencies are installed in the project environment *prior* to building the package itself.

For example, historically, to install `cchardet` without build isolation, you would first need to install the `cython` and `setuptools` packages in the project environment, followed by a separate invocation to install `cchardet` without build isolation:

```
$ uv venv
$ uv pip install cython setuptools
$ uv pip install cchardet --no-build-isolation
```

uv simplifies this process by allowing you to specify packages that should not be built in isolation via the `no-build-isolation-package` setting in your `pyproject.toml` and the `--no-build-isolation-package` flag in the command line. Further, when a package is marked for disabling build isolation, uv will perform a two-phase install, first installing any packages that support build isolation, followed by those that do not. As a result, if a project's build dependencies are included as project dependencies, uv will automatically install them before installing the package that requires build isolation to be disabled.

For example, to install `cchardet` without build isolation, include the following in your `pyproject.toml`:

```toml title="pyproject.toml"
[project]
name = "project"
version = "0.1.0"
description = "..."
readme = "README.md"
requires-python = ">=3.12"
dependencies = ["cchardet", "cython", "setuptools"]

[tool.uv]
no-build-isolation-package = ["cchardet"]
```

```
When running `uv sync`, uv will first install `cython` and `setuptools` in the
project environment,
followed by `cchardet` (without build isolation):
```

```console
$ uv sync --extra build
 + cchardet==2.1.7
 + cython==3.1.3
 + setuptools==80.9.0
```

Similarly, to install `flash-attn` without build isolation, include the following in your `pyproject.toml`:

```toml title="pyproject.toml"
[project]
name = "project"
version = "0.1.0"
description = "..."
readme = "README.md"
requires-python = ">=3.12"
dependencies = ["flash-attn", "torch"]

[tool.uv]
no-build-isolation-package = ["flash-attn"]
```

```
When running `uv sync`, uv will first install `torch` in the project environment,
followed by
`flash-attn` (without build isolation). As `torch` is both a project dependency and a
build
dependency, the version of `torch` is guaranteed to be consistent between the build
and runtime
environments.
```

```
A downside of the above approach is that it requires the build dependencies to be
installed in the
project environment, which is appropriate for `flash-attn` (which requires `torch`
both at
build-time and runtime), but not for `cchardet` (which only requires `cython` at
build-time).

To avoid including build dependencies in the project environment, uv supports a two-
step
installation process that allows you to separate the build dependencies from the
packages that
require them.

For example, the build dependencies for `cchardet` can be isolated to an optional
`build` group, as
in:

```toml title="pyproject.toml"
[project]
name = "project"
version = "0.1.0"
description = "..."
readme = "README.md"
requires-python = ">=3.12"
dependencies = ["cchardet"]

[project.optional-dependencies]
build = ["setuptools", "cython"]

[tool.uv]
no-build-isolation-package = ["cchardet"]
```

Given the above, a user would first sync with the `build` optional group, and then without it to remove the build dependencies:

```
$ uv sync --extra build
 + cchardet==2.1.7
 + cython==3.1.3
 + setuptools==80.9.0
$ uv sync
 - cython==3.1.3
 - setuptools==80.9.0
```

Some packages, like `cchardet`, only require build dependencies for the *installation* phase of `uv sync`. Others require their build dependencies to be present even just to resolve the project's dependencies during the *resolution* phase.

In such cases, the build dependencies can be installed prior to running any `uv lock` or `uv sync` commands, using the lower lower-level `uv pip` API. For example, given:

```toml title="pyproject.toml" [project] name = "project" version = "0.1.0" description = "..." readme = "README.md" requires-python = ">=3.12" dependencies = ["flash-attn"]

[tool.uv] no-build-isolation-package = ["flash-attn"]


You could run the following sequence of commands to sync `flash-attn`:
```
```

```console
$ uv venv
$ uv pip install torch setuptools
$ uv sync
```

Alternatively, users can instead provide the `flash-attn` metadata upfront via the `dependency-metadata` setting, thereby forgoing the need to build the package during the dependency resolution phase. For example, to provide the `flash-attn` metadata upfront:

```toml title="pyproject.toml" [[tool.uv.dependency-metadata]] name = "flash-attn" version = "2.6.3" requires-dist = ["torch", "einops"]
```

## Editable mode

By default, the project will be installed in editable mode, such that changes to the source code are immediately reflected in the environment. `uv sync` and `uv run` both accept a `--no-editable` flag, which instructs uv to install the project in non-editable mode. `--no-editable` is intended for deployment use-cases, such as building a Docker container, in which the project should be included in the deployed environment without a dependency on the originating source code.

## Conflicting dependencies

uv resolves all project dependencies together, including optional dependencies ("extras") and dependency groups. If dependencies declared in one section are not compatible with those in another section, uv will fail to resolve the requirements of the project with an error.

uv supports explicit declaration of conflicting dependency groups. For example, to declare that the `optional-dependency` groups `extra1` and `extra2` are incompatible:

```toml title="pyproject.toml" [tool.uv] conflicts = [ [ { extra = "extra1" }, { extra = "extra2" }, ], ]
```

Or, to declare the development dependency groups `group1` and `group2` incompatible:

```toml title="pyproject.toml" [tool.uv] conflicts = [ [ { group = "group1" }, { group = "group2" }, ], ]
```

See the resolution documentation for more.

## Limited resolution environments

If your project supports a more limited set of platforms or Python versions, you can constrain the set of solved platforms via the `environments` setting, which accepts a list of PEP 508 environment markers. For example, to constrain the lockfile to macOS and Linux, and exclude Windows:

```toml title="pyproject.toml" [tool.uv] environments = [ "sys_platform == 'darwin'", "sys_platform == 'linux'", ]
```

See the resolution documentation for more.

## Required environments

If your project *must* support a specific platform or Python version, you can mark that platform as required via the `required-environments` setting. For example, to require that the project supports Intel macOS:

```toml title="pyproject.toml" [tool.uv] required-environments = [ "sys_platform == 'darwin' and platform_machine == 'x86_64'", ]
```

The `required-environments` setting is only relevant for packages that do not publish a source distribution (like PyTorch), as such packages can *only* be installed on environments covered by the set of pre-built binary distributions (wheels) published by that package.

See the resolution documentation for more.